

SEUPD@CLEF: Team retrix on Source Retrieval for Scientific Web Claims

Fabio Baldan¹, Davide Donati¹, Alvisè Garberino¹, Giancarlo Padoan¹ and Marco Tessari¹

¹University of Padua, Italy

Abstract

We present the results of the retrix group for the CheckThat! 2026 - Task 1, in which the goal is to identify the scientific paper implicitly referenced in a social media post. The input consists of short, noisy and usually informal posts that mention scientific findings without providing explicit links or explicit citations.

Our approach is designed to operate in a multilingual setting, as the dataset includes posts in English, German, and French. To handle this, we used multi-lingual language models.

DA FINIRE CON PIPELINE FINALEEEEEEE DA FINIRE CON PIPELINE FINALEEEEEEE DA FINIRE CON PIPELINE FINALEEEEEEE DA FINIRE CON PIPELINE FINALEEEEEEE

Keywords

Fact Checking, Information Retrieval, Scientific Document Retrieval, Social media fact verification, CLEF, Neural Re-ranking, Cross Encoder

1. Introduction

The rapid spread of scientific claims through social media creates a difficult fact-checking scenario: posts often report, simplify, or sensationalize research findings while omitting the original source. In many cases, the cited study is not linked explicitly with a URL or a bibliographic reference, but only mentioned indirectly through fragments such as the reported finding, the venue, the institution, or the authors. As a consequence, verifying such claims requires first identifying the scientific publication that is being implicitly referenced.

This problem is the focus of Task 1 of the CLEF 2026 CheckThat! Lab, *Source Retrieval for Scientific Web Claims*. Given a social media post containing a scientific claim and an implicit reference to a paper, the goal is to retrieve the correct publication from a fixed collection of 10.000 candidate papers written in English.

The task is challenging for several reasons. First, the linguistic gap between social media and scientific writing is substantial: posts are short, informal and often contain paraphrases, slogans, hashtags or incomplete statements, whereas scientific papers use more precise and technical language. Second, the reference to the source paper is implicit rather than explicit so the retrieval system must infer the connection from partial semantic evidence rather than from direct citations. Third, relevant clues may appear in different forms, such as mentions of a scientific result, a medical outcome, a research institution, or a publication venue, requiring the system to combine lexical and semantic evidence effectively.

An additional challenge is multilinguality: the official query sets include English, German and French posts while the candidate pool remains shared across languages.

Team Retrix approaches this problem with a three-stage sequential pipeline.

1. Raw tweet-style queries are enriched via pseudo-relevance feedback: BAAI/bge-large-en-v1.5 encodes both queries and corpus documents into a shared embedding space, retrieves the top-5

“Search Engines”, course at the master degree in “Computer Engineering”, Department of Information Engineering, and at the master degree in “Data Science”, Department of Mathematics “Tullio Levi-Civita”, University of Padua, Italy. Academic Year 2025/2026

✉ fabio.baldan.1@studenti.unipd.it (F. Baldan); davide.donati@studenti.unipd.it (D. Donati);
alvisè.garberino@studenti.unipd.it (A. Garberino); giancarlo.padoan@studenti.unipd.it (G. Padoan);
marco.tessari.8@studenti.unipd.it (M. Tessari)



© 2026 Copyright for this paper by its authors.
Use permitted under Creative Commons License Attribution 4.0 International (CC BY 4.0).

semantically similar papers per query and extracts the most frequent terms from those papers as expansion tokens. Not-english queries are processed with BAAI/bge-m3 with the same high-level principles. Not-english queries are translated in English and then processed like the english ones.

2. The expanded queries are then fed into BAAI/bge-m3 operating in hybrid mode, which combines dense inner-product search via FAISS [1] with sparse lexical matching and ColBERT multi-vector scoring to produce an initial ranked list of candidates. The goal here is to have the highest recall possible.
3. Finally, the top candidates are reranked by a neural cross-encoder, in our case nvidia/llama-nemotron-rerank-1b-v2. in our case nvidia/llama-nemotron-rerank-1b-v2 fine-tuned on XXXXXXXX in our case answerdotai/ModernBERT-large fine-tuned on XXXXXXXXXX

Throughout the project we also investigated the effect of fine-tuning the reranker on domain-specific hard negatives extracted from the training set.

Evaluated on the CheckThat! 2026 Task 1 dataset our system ranked XXXXXXXXXXXX on the official leaderboard attaining an average Mean Reciprocal Rank at 5 (MRR@5) of XXXXXXXXXXXX % on the development set (XXXXX place).

The paper is organized as follows: Section 2 discusses the most relevant research directions related to this task; Section 4 describes our approach; Section 5 explains our experimental setup; Section 6 discusses our main findings; finally, Section 7 draws some conclusions and outlooks for future work.

2. Related Work

The task addressed in this paper lies at the intersection of fact-checking and information retrieval. A natural starting point is lexical retrieval, where probabilistic ranking models such as BM25 remain strong baselines thanks to their efficiency, interpretability and robustness in candidate generation [2]. In the setting of scientific claim source retrieval, lexical matching is especially useful when a post mentions recognizable entities or terminology that also appears in the title, abstract, venue or author list of the referenced paper.

Neural retrieval methods have been proposed to mitigate the lexical mismatch between queries and documents by learning dense semantic representations. More broadly, neural information retrieval has shifted attention from exact token overlap to representation learning and learned matching functions [3]. In particular, dense dual-encoder architectures such as Dense Passage Retrieval (DPR) demonstrated that learned query and document embeddings can be highly effective for first-stage retrieval [4]. This line of work is especially relevant to our task because social media posts often paraphrase scientific findings instead of reusing the exact wording found in the original publication.

Between initial retrieval and final ranking, many modern systems adopt multi-stage pipelines that trade off efficiency and effectiveness. Late-interaction models such as ColBERT preserve fine-grained token-level matching while remaining substantially more scalable than fully pairwise neural scoring over the whole collection [5]. Likewise, transformer-based rerankers have become a common second-stage component for refining an initial candidate set with stronger query-document interactions [6]. These approaches motivate retrieval architectures that first generate a manageable set of candidates and then apply more expensive relevance estimation only where it is most useful.

Se fine tuning va bene è da aggiungere parte su hard negatives e ricerca embedding incrociati usate: <https://arxiv.org/html/2505.18366v1> <https://huggingface.co/blog/train-reranker>

Our work is also related to the broader literature on automatic claim verification and, more specifically, to the CheckThat! shared tasks, which have addressed the detection, verification, and analysis of claims in social media across several editions [7, 8, 9]; however, source retrieval for scientific web claims differs from standard verification tasks in an important way: the system is not asked to classify a claim as true or false, but to recover the original scientific paper implicitly referenced in the post and for this reason, the task is closely connected to evidence retrieval, while also requiring robust matching between informal multilingual posts and formal scientific documents.

3. Dataset

The dataset used in this work is organized as a document retrieval task: given a short social-media post that implicitly refers to a scientific result, the system must identify the corresponding paper from a fixed candidate pool. In the version available in our project, the data are divided into a shared document collection (`code/data/collection_data.json`) and language-specific query files for English, German, and French; this setting is inherently multilingual on the query side, while the candidate pool is shared across all languages.

The document collection contains 10,000 scientific papers; each entry is identified by a unique pubkey and includes `title`, `abstract`, `venue`, and `authors`; in our pipeline, each paper is represented by the concatenation of title and abstract, which provides a compact but informative document view for both retrieval and reranking. The collection is not split by language, so all queries, regardless of their language, must be matched against the same set of candidate papers.

The raw query set is organized into multiple JSON files under `code/data/Train_set/`, `code/data/Dev_set/`, and `code/data/Test_set/`, each containing the text of the social media post together with its identifier (`index`); in the training and development portions of the dataset, each query is further paired with the gold pubkey of the scientific paper implicitly referenced in the post, which enables supervised model development and internal evaluation. By contrast, the public test files provide only the query identifiers and post texts, with the relevance labels withheld in accordance with the official evaluation protocol. Table 1 summarizes the size of the data splits used in our experiments; while training data are available for English, German, and French, the public development portion included in the repository covers only German and French.

Table 1: Dataset statistics for the version used in our experiments.

Split	EN	DE	FR	Total
Train	14,977	1,460	2,807	19,244
Dev	–	386	702	1,088
Test	6,076	876	1,220	8,172

One of the main challenges of the dataset lies in the marked mismatch between queries and documents: the social-media posts are short and noisy, averaging roughly 29–34 words in the labeled splits, whereas the document representations obtained by concatenating title and abstract are substantially longer, with an average length of about 245 words. At the same time, the relation between queries and papers is many-to-one, since the labeled training and development splits contain 20,332 queries but only 5,458 distinct referenced papers, which means that multiple posts may paraphrase, summarize, or discuss the same publication from different perspectives. Taken together, these characteristics make the benchmark particularly suitable for evaluating retrieval systems that must integrate lexical evidence, semantic similarity, and cross-lingual matching.

4. Methodology

We started from a lexical baseline based on Apache Lucene BM25 and progressively moved to a multi-stage neural pipeline. Early experiments focused on analyzer choices, stopwords lists, and BM25 variants;

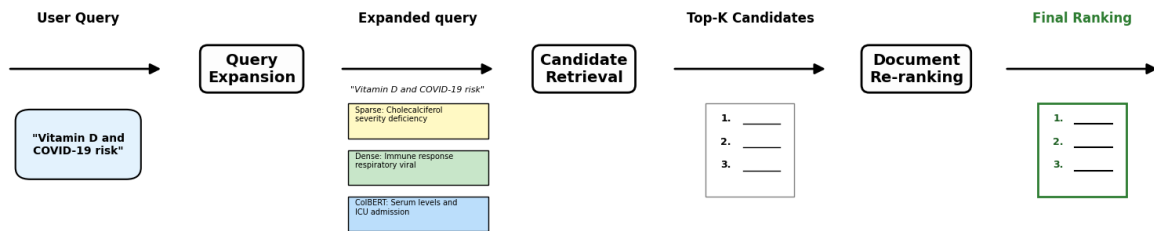


Figure 1: Three-stage pipeline for information retrieval.

these runs were useful to establish a stable baseline, but they also showed that lexical overlap alone is insufficient for this task.

We therefore adopted a three-stage architecture that separates high-recall candidate generation from expensive neural ranking.

We implemented a three-stage pipeline (Figure 1) for our information retrieval system, consisting of:

- **Query expansion:** We expanded the original query with additional context to bridge the gap between informal and scientific language.
- **Retrieval:** We retrieved relevant documents from the corpus using the expanded query.
- **Reranking:** We reranked the retrieved documents based on their relevance to the original query.

For each of these stages, we experimented with different models, choosing the ones that performed better in relation to the others.

4.1. Query Expansion

The first step in our pipeline is query expansion: given the original query, we generate extra terms and tokens related to it. This serves two purposes: to increase the number of available tokens for a match, and to bridge the gap between the original informal query and the formal academic language used in the document corpus.

Originally, we experimented with PRF-style expansion using semantic retrieval with BAAI/bge-large-en-v1.5. We then explored different methods for expansion, including Pseudo-Relevance Feedback (PRF) and similarity with static word embeddings using Word2Vec, testing these on both the original and the preliminary expanded queries.

In the final architecture, we used a multilingual LLM-based expansion script (`query_expansion_multilingual.py`) default model `meta-llama/Llama-3.2-1B`) to generate multiple retrieval-oriented views of each query. The model is prompted to produce distinct representations, tailored to our multi-stage retrieval needs:

- *sparse*: A list of domain-specific keywords and technical synonyms (e.g., scientific nomenclature) designed to maximize lexical overlap and mitigate the vocabulary mismatch in BM25-based retrieval.
- *dense*: A semantically enriched re-phrasing of the claim that provides a broader context, allowing the bi-encoder in the next step of the architecture to produce more robust embeddings in the high-dimensional vector space.
- *ColBERT*: A structured, formal version of the claim (e.g., “Serum levels and ICU admission”) that provides coherent token sequences for fine-grained late-interaction scoring.

This multi-view expansion ensures that the downstream hybrid retriever can leverage both high-precision lexical matches and broad semantic overlaps. By generating these representations simultaneously within a single inference step of the same LLM, we maintain a coherent relationship between the

different query “views”, effectively providing a unified but multi-faceted context for the retrieval pipeline.

The resulting expanded queries are then fed into the multi-stage retrieval framework described next.

4.2. Retrieval

The retrieval stage is designed as a high-recall candidate generator between query expansion and neural reranking. Its objective is not to make the final relevance decision, but to ensure that the true source paper is included in a sufficiently large candidate set for the subsequent, more expensive cross-encoder stage. For this reason, after experimenting with a Lucene BM25 baseline, BGE-M3 only on dense vectors, the final pipeline uses the hybrid BGE-M3 retriever.

This choice reflects the nature of the task: social media claims often contain informal paraphrases, hashtags, named entities, numbers, and fragments of scientific terminology, so relying only on exact lexical overlap or only on dense semantic similarity would discard useful evidence.

The retriever searches the fixed paper collection provided by the task organizers and named `collection_data.json`.

Each paper is represented at document level by the concatenation of its title and abstract, and the retrieved unit is always the paper pubkey. We did not split papers into smaller chunks because the task requires identifying the source publication rather than retrieving an evidence passage, and using one representation per paper avoids an additional chunk-to-document aggregation step. Although the collection also contains metadata such as venue and authors, the hybrid retriever uses only title and abstract, which are the fields consistently encoded for all documents.

The input queries are the multilingual expanded queries stored in `expanded_queries_multilingual_merged.js` produced by the expansion stage described above.

The expansion step provides three views of the same information need: an embedding view for semantic matching, a sparse view for lexical and terminology-based evidence, and a colbert view for fine-grained token interaction. BGE-M3 is well suited to this setting because it can produce dense vectors, sparse lexical weights, and ColBERT-style multi-vector representations from the same multilingual encoder. Thus, the retriever can combine broad semantic matching with exact term evidence and token-level matching without switching between unrelated encoders.

Candidate selection is performed in two steps. First, all documents are encoded as dense BGE-M3 vectors and searched with an inner-product FAISS index [1]. This dense stage is used as an efficient broad filter over the whole collection. The default setting retrieves a large pool, `candidate_multiplier` $\times$ `top_k`, with `candidate_multiplier=3` and `top_k=1000`, before the final truncation. This deliberately large candidate set favours recall, which is more important at this stage than precision. Second, the dense candidates are rescored using the sparse and ColBERT representations. The sparse component rewards important overlapping terms, while the ColBERT-style component captures local token correspondences that may be lost in a single dense vector [5].

The final configuration was selected through a sequence of runs recorded during development. We first tuned the lexical baseline by varying the Lucene analyzer, stop lists, stemming strategy, and BM25 searcher variants. These experiments improved the classical baseline but also showed its limitation for this task: even with query expansion and reranking, BM25-based candidate generation remained more dependent on surface overlap than desired. We then tested dense retrieval with BGE models, which improved the semantic coverage of the candidate set, especially for paraphrased or multilingual claims. The best retrieval-only results in the analysis came from the BGE-M3 hybrid variants, confirming the motivation for combining semantic, lexical, and token-interaction evidence in the same first-stage retriever.

Among the hybrid configurations, the run labelled `bge_m3_hybrid_513` was retained because it gave the strongest recall-oriented trade-off among the tested no-rerank retrievers: it used weights 0.5/0.15/0.35 for dense, sparse, and ColBERT scoring, respectively, with multiplier 3 and maximum hybrid length 384. A closely related configuration with a larger ColBERT weight obtained a nearly identical MRR@5 but lower Recall@100, so we preferred the setting that better matched the role of

retrieval as a candidate generator. This choice was further confirmed by downstream runs: reranking the candidates from `bge_m3_hybrid_513` consistently outperformed the corresponding BM25-based reranked runs, and increasing the reranked candidate depth from a few hundred documents to 500–1000 documents improved the final recall available to the reranker.

We also experimented with a learned fusion strategy for these hybrid signals. In particular, the experimental test performed includes a LightGBM LambdaMART option that builds features from the dense, sparse, and ColBERT scores, together with normalized score variants and dense-rank features, and then learns how to combine them. However, the retained retrieval configuration uses the simpler linear fusion, which was easier to reproduce and aligned well with the recall-oriented role of this stage. The score used in the final hybrid retriever is therefore:

$$s(q, d) = 0.5 s_{\text{dense}}(q, d) + 0.15 s_{\text{sparse}}(q, d) + 0.35 s_{\text{colbert}}(q, d).$$

The largest weight is assigned to the dense score because it provides the main multilingual semantic signal; the sparse and ColBERT terms add complementary lexical and fine-grained matching evidence. Candidates are sorted by this score in descending order and truncated to the final `top_k`. The retrieval output is a JSON mapping from query id to an ordered list of paper pubkeys. Scores are not exported, because the next stage treats retrieval as a candidate generator and recomputes relevance with a cross-encoder over the returned document identifiers.

4.3. Reranking

The reranking stage receives the candidate list produced by the hybrid retriever and reorders it with cross-encoders. Its goal is to improve early precision (especially `MRR@5`) while preserving as much recall as possible from the first stage.

Our strongest single-stage reranker is `nvidia/llama-nemotron-rerank-1b-v2`. For each query–document pair, we build an input sequence with the template question: `{q}` followed by `passage: {p}` and compute a scalar relevance score with a sequence-classification head. Documents are then sorted by this score. In practice, reranking deeper candidate pools (e.g., 500–1000 documents) gave better final results than shallow pools, confirming the importance of high recall before neural ranking.

We also evaluated alternative rerankers, including Qwen3 and GTE-ModernBERT variants, and tested cascaded reranking. In the best cascaded configuration, Nemotron is used as the first reranking pass and a second reranker (`BAAI/bge-reranker-v2-m3` or a fine-tuned `ModernBERT-large`) refines the top portion of the list. This two-step reranking improves top-rank ordering over single-stage reranking.

4.4. Evaluation

We evaluate systems on the CheckThat! Task 1 query sets by matching retrieved pubkeys against the gold pubkey for each query. The official optimization target is `MRR@5`, which we use as the primary score during model and parameter selection. In parallel, we track `Recall@100` to monitor candidate-generation quality and to verify that reranking gains do not come from overly aggressive filtering.

For internal analysis, our evaluator also computes additional ranking metrics (e.g., `Recall@1/5/10/100`, `Precision@1/5/10`, `MAP`, and `nDCG@k`). This broader view is useful to interpret trade-offs between recall-oriented retrieval settings and precision-oriented reranking settings, while keeping `MRR@5` as the main criterion for reporting and model selection.

5. Experimental Setup

We use the official CheckThat! 2026 Task 1 collection format: social-media claims as queries and a fixed paper corpus in `code/data/collection_data.json`. Query files are handled in JSON format (English, German, and French) and expanded versions are generated in `code/data/expanded_queries_multilingual`. Retrieved outputs are always ranked lists of paper pubkeys.

Table 2
Representative runs selected from the experimental campaign.

Family	Run label	Key configuration	MRR@5	Recall@100
Analyzer tuning	Analyzer V2	Lexical retrieval with MyEnglishAnalyzer_v2 and BGE-V1 expansion	0.503	0.796
Analyzer tuning	Analyzer V1.12	Lexical retrieval with KStem and custom stoplist variant	0.450	0.783
Retrieval only	Hybrid A	BGE-M3 hybrid retrieval, dense 0.35, sparse 0.15, ColBERT 0.50	0.557	0.837
Retrieval only	Hybrid B	BGE-M3 hybrid retrieval, dense 0.50, sparse 0.15, ColBERT 0.35	0.557	0.839
Single-stage reranking	Nemotron top-400	Hybrid retrieval followed by Nemotron reranking on 400 candidates	0.694	0.893
Single-stage reranking	Nemotron top-1000	Hybrid retrieval followed by Nemotron reranking on 1000 candidates	0.702	0.911
Alternative rerankers	Qwen3-4B	Hybrid retrieval followed by Qwen3 reranking	0.662	0.839
Alternative rerankers	GTE-ModernBERT	Hybrid retrieval followed by GTE-ModernBERT reranking	0.652	0.852
Cascaded reranking	Nemotron + BGE-v2-M3	Nemotron first-stage reranking followed by BGE reranking	0.741	0.893
Cascaded reranking	Nemotron + ModernBERT-FT	Nemotron first-stage reranking followed by fine-tuned ModernBERT reranking	0.766	0.893

Our primary evaluation measure is MRR@5, consistent with the task objective and the official scorer used in `results/scoring_official/`. During development, we also monitor Recall@100 and additional internal metrics from the Java evaluator (Recall@k, Precision@k, MAP, nDCG@k) to analyze recall/precision trade-offs across retrieval and reranking configurations.

The implementation is organized into: (i) Java components for indexing, baseline retrieval, and evaluation (`code/src/main/java/unipd/se/`); (ii) Python components for multilingual query expansion, hybrid BGE-M3 retrieval, and neural reranking (`code/src/main/java/unipd/se/py/`); and (iii) run outputs and metrics under `results/`. The project repository is available at <https://bitbucket.org/upd-dei-stud-prj/seupd2526-retrix.git>.

In terms of execution environment, lightweight preprocessing and analysis can run on CPU, while the neural stages (LLM-based expansion, hybrid BGE-M3 encoding, and cross-encoder reranking) are configured to use CUDA GPUs when available, with automatic CPU fallback paths implemented in the code.

6. Results and Discussion

6.1. Main Results

Table 2 summarizes ten representative runs selected from our experimental campaign; rather than listing the top-scoring runs only, we chose configurations that cover the main families of experiments explored during the project, namely analyzer tuning, retrieval-only hybrid search, single-stage reranking, alternative rerankers, and cascaded reranking.

The results in Table 2 show a clear progression across the different stages of the project. The best retrieval-only configuration reaches an MRR@5 of 0.557, while the best cascaded system reaches 0.766. The largest individual improvement comes from adding `nvidia/llama-nemotron-rerank-1b-v2` as the first reranking stage, which moves the system from the 0.557 range of the hybrid-only runs to 0.702. A second reranking stage then brings a further gain, with the fine-tuned ModernBERT model producing the strongest final configuration.

6.2. Ablation / Run Analysis

Analyzer tuning: We initially explored stronger English analyzers and several stemming and stop-word variants in order to improve lexical matching between short social-media posts and scientific abstracts. This family peaks at an MRR@5 of 0.503 with `MyEnglishAnalyzer_v2`, while the other analyzer variants remain around the 0.45–0.50 range. These runs were useful in validating the pre-processing choices, but they also showed that text normalization alone is not enough to bridge the semantic gap between informal claims and academic documents. Analyzer tuning yields measurable improvements, but its impact is limited when compared with semantic retrieval and neural reranking.

Hybrid retrieval without reranking: A shift from analyzer-focused experiments to hybrid retrieval leads to a substantially larger improvement; the two best retrieval-only systems, reported in Table 2 as Hybrid A and Hybrid B, both reach an MRR@5 of about 0.557, with recall@100 between 0.837 and 0.839. The difference between them lies in the balance between the dense and ColBERT components, while the sparse contribution is kept fixed. This indicates that combining dense, sparse, and ColBERT-style signals already produces a strong candidate set before any reranking is applied; at the same time the very small difference between the two configurations suggests that, once the hybrid formulation is in place, the exact internal weighting matters less than the transition away from purely lexical retrieval. Hybrid retrieval is the first major step change in effectiveness and provides the strongest retrieval-only foundation for the full pipeline.

Single-stage reranking with Nemotron: Adding `nvidia/llama-nemotron-rerank-1b-v2` on top of the hybrid candidates produces the largest isolated gain in MRR@5; the representative Nemotron runs increase from the 0.557 range of the retrieval-only systems to 0.694 and 0.702, while recall@100 also rises from about 0.839 to as high as 0.911. We also observe a limited but consistent benefit from increasing the number of candidates exposed to the reranker: moving from the top-400 to the top-1000 setting slightly improves both MRR@5 and recall@100; this suggests that the reranker can exploit a broader candidate pool without being significantly affected by additional noise. The first reranking stage is the most impactful component of the pipeline, and a larger candidate pool brings further, although diminishing, gains.

Alternative rerankers: To verify that the gains were not tied to a single model, we also evaluated other rerankers, including `Qwen3-reranker-4B` and `Alibaba-NLP/gte-reranker-modernbert-base`; both models clearly outperform the retrieval-only baseline in terms of MRR@5, reaching values of 0.662 and 0.652, respectively. However, they remain below the Nemotron runs, especially on MRR@5, which suggests weaker ranking quality in the first positions of the final list; even so these experiments confirm that the hybrid retriever produces candidate sets that can be improved by different neural rerankers. Alternative rerankers validate the overall pipeline design, but Nemotron is the strongest first-stage reranker among the models we tested.

Cascaded reranking: Our best results are obtained by applying a second reranking stage to the candidates already filtered by Nemotron; the cascade based on `BAAI/bge-reranker-v2-m3` reaches an MRR@5 of 0.741, while the best overall system, based on a fine-tuned version of `answerdotai/ModernBERT-large`, reaches 0.766. Both cascaded runs keep recall@100 at 0.893, which is slightly lower than the Nemotron top-1000 configuration but still substantially higher than the retrieval-only systems; this indicates that the second reranker mainly improves the ordering of already strong candidates rather than increasing the breadth of the candidate pool. Cascaded reranking is the best strategy to maximize early precision, and the fine-tuned ModernBERT reranker provides the strongest final ranking.

7. Conclusions and Future Work

Our experiments show that the strongest improvement comes from neural reranking, especially when a hybrid retriever is followed by `nvidia/llama-nemotron-rerank-1b-v2` and then refined by a second reranking stage based on fine-tuned ModernBERT. In contrast, analyzer tuning and other lexical adjustments produced only limited gains when compared with the impact of hybrid retrieval and multi-stage reranking. Future work will focus on improving the multilingual side of the pipeline and on further refining the second-stage reranker.

8. Group Members Contribution

Fabio Baldan has contributed to this project by:

1. Helping designing and implementing the initial BM25-basic pipeline;
2. Writing and revising some chapters of the documentation based on the actual code and on the concepts learned during the project development;
3. Tracking and reviewing the annotation-part for the runs with various systems;
4. Trying various models for the re-rank part to enhance the MRR@5;
5. Implementing a custom evaluator to focus on metrics that matter the most;
6. Reviewing and improving the query-expansion;
7. Fine-tuning the re-reranker to improve the MRR@5 and assure an advantage if compared with the default one.

Davide Donati has contributed to this project by:

1. Helping designing and implementing the initial BM25 basic pipeline;
2. Implementing a BooleanQuery-based parsing in `Searcher.java` to separate the original tweet and expanded terms into different clauses with tailored boosts (and applying field filters for venue/authors), improving retrieval precision;
3. Implementing and testing the multilingual query-expansion pipeline (e.g. `query_expansion_multilingual` tested with `PROVEDAVIDE.py`) and fixing issues found during integration;
4. Adding a PRF (pseudo-relevance feedback) implementation and accompanying AI-assisted documentation/examples (PRF code is available in the discarded-code area for reference and analysis);
5. Contributing to documentation and write-ups;
6. Keeping track of experimental runs and results for the systems he developed and tested.

Alvise Garberino has contributed to this project by:

1. Helping design and implement the initial BM25-based retrieval pipeline;
2. Helping and searching the best models and techniques to implement a re-ranker model;
3. Implementing and improving the re-ranker fine-tuning pipeline, including data augmentation through back-translation, hard-negative construction/mining, and data-distillation support;
4. Contributing to the evaluation workflow by tracking experimental runs, analyzing results, and supporting checkpoint evaluation;
5. Contributing to the Java support code and documentation, including refactoring/query-model improvements and JavaDoc updates;
6. Writing and revising parts of the documentation, especially the introduction, related work, results, conclusion, and setup sections, with appropriate academic references.

Giancarlo Padoan has contributed to this project by:

Table 3
Frequency of Special Characters

Non-English or Math	Frequency	Comments
Ø	1 in 1,000	For Swedish names
π	1 in 5	Common in math
\$	4 in 5	Used in business
Ψ_1^2	1 in 40,000	Unexplained usage

1. Designing, implementing and fixing the initial basic pipeline;
2. Writing and correcting some chapters of the documentation based on the actual code and on the notions learned while building the project;
3. Designing and implementing an hybrid-retrieval system to enhance recall, which is the core of the final architecture;
4. Trying various models for the embedding-representation part to enhance the MRR@5, ;
5. Implementing the first version of query-expansion functionality, and improving it by testing different techniques;
6. Tested different scoring function for combining the hybrid-retrieval scores.
7. Testing various cross-encoder models for the re-ranking stage of the pipeline.
8. Keeping track of the results of the systems that he tried.

Marco Tessari has contributed to this project by:

1. Helping designing and implementing the initial BM25-basic pipeline;
2. Improved the performance of the English Analyzer (text-processing, stemmer, filters and stopwords);
3. Writing part of the docs and helping other members by reviewing their writes;
4. Keeping track of the results of the systems that he tried;
5. Helping and searching the best models and techniques to implement an embedding model;
6. Experimenting with word2vec static embeddings for query expansion
7. Helping to find the best weights to enhance recall in the hybrid-retrieval system.

9. Misc [TO BE REMOVED]

INTRODUCTION.TEX E RELATED.TEX SONO DA FINIRE QUANDO AVREMO I RISULTATI COMPLETI (VEROSIMILMENTE QUALCHE GIORNO PRIMA DELLA CONSEGNA) NON CANCELLARE QUESTA FRASE E NON MODIFICARE QUEI FILE (SE NON PER AGGIUNGERE COSE UTILI) SENNÒ MI DIMENTICO DI FARLO E PRENDIAMO 5/20 GRAZIE :) <3

9.1. Tex Files

Put your $\text{\LaTeX}$ files into the `section` folder as shown in the examples above.

9.2. Figures

Put your figures into the `figure` folder and put the caption under the figure. Example of reference to Figure 2.

9.3. Tables

Put the caption above the table. Example of reference to Table 3.

See the `booktab` packaged documentation for further options.



Figure 2: 1907 Franklin Model D roadster. Photograph by Harris & Ewing, Inc. [Public domain], via Wikimedia Commons. (<https://goo.gl/VLCRBB>).

9.4. Bibliography

Example of citations:

- name: Salton [10]
- parenthesis: [10]

An initial list of references is provided in the files `bibliography.bib` and `proceedings.bib` that you can expand.

See the `natbib` packaged documentation for further options.

9.5. Acronyms

Use the

`\ac{acronym}`

command to insert acronyms, eg. *Average Precision (AP)*. The command will expand the acronym the first time it is used.

An initial list of acronyms is provided in the file `acronyms.tex` that you can expand.

See the `acronym` packaged documentation for further options.

References

- [1] Faiss, Faiss documentation, <https://faiss.ai/>, ????. Accessed: 01-05-2026 (DD-MM-YYYY, italian date format).
- [2] S. E. Robertson, U. Zaragoza, The Probabilistic Relevance Framework: BM25 and Beyond, *Foundations and Trends in Information Retrieval (FnTIR)* 3 (2009) 333–389.

- [3] S. Marchesin, A. Purpura, G. Silvello, Focal elements of neural information retrieval models. An outlook through a reproducibility study, *Information Processing & Management* 57 (2020) 102109:1–102109:29.
- [4] V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, W.-t. Yih, Dense Passage Retrieval for Open-Domain Question Answering, in: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Association for Computational Linguistics, Online, 2020, pp. 6769–6781. URL: <https://aclanthology.org/2020.emnlp-main.550/>. doi:10.18653/v1/2020.emnlp-main.550.
- [5] O. Khattab, M. Zaharia, ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT, in: *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR 2020)*, ACM Press, New York, USA, 2020, pp. 39–48. URL: <https://doi.org/10.1145/3397271.3401075>. doi:10.1145/3397271.3401075.
- [6] R. Nogueira, Z. Jiang, R. Pradeep, J. Lin, Document Ranking with a Pretrained Sequence-to-Sequence Model, in: *Findings of the Association for Computational Linguistics: EMNLP 2020*, Association for Computational Linguistics, Online, 2020, pp. 708–718. URL: <https://aclanthology.org/2020.findings-emnlp.63/>. doi:10.18653/v1/2020.findings-emnlp.63.
- [7] T. Elsayed, P. Nakov, A. Barrón-Cedeño, M. Hasanain, R. Suwaileh, G. Da San Martino, P. Atanasova, Overview of the CLEF-2019 CheckThat!: Automatic Identification and Verification of Claims, in: F. Crestani, M. Braschler, J. Savoy, A. Rauber, H. Müller, D. E. Losada, G. Heinatz Bürki, L. Cappellato, N. Ferro (Eds.), *Experimental IR Meets Multilinguality, Multimodality, and Interaction. Proceedings of the Tenth International Conference of the CLEF Association (CLEF 2019)*, Lecture Notes in Computer Science (LNCS) 11696, Springer, Heidelberg, Germany, 2019, pp. 301–321.
- [8] A. Barrón-Cedeño, T. Elsayed, P. Nakov, G. Da San Martino, M. Hasanain, R. Suwaileh, F. Haouari, N. Babulkov, B. Hamdan, A. Nikolov, S. Shaar, Z. Sheikh Ali, Overview of CheckThat! 2020: Automatic Identification and Verification of Claims in Social Media, in: A. Arampatzis, E. Kanoulas, T. Tsikrika, S. Vrochidis, H. Joho, C. Lioma, K. Eickhoff, A. Névél, L. Cappellato, N. Ferro (Eds.), *Experimental IR Meets Multilinguality, Multimodality, and Interaction. Proceedings of the Eleventh International Conference of the CLEF Association (CLEF 2020)*, Lecture Notes in Computer Science (LNCS) 12260, Springer, Heidelberg, Germany, 2020, pp. 215–236.
- [9] P. Nakov, G. Da San Martino, T. Elsayed, A. Barrón-Cedeño, R. Míguez, S. Shaar, F. Alam, F. Haouari, M. Hasanain, W. Mansour, B. Hamdan, Z. S. Ali, N. Babulkov, A. Nikolov, G. K. Shahi, J. M. Struß, T. Mandl, M. Kutlu, Y. S. Kartal, Overview of the CLEF-2021 CheckThat! Lab on Detecting Check-Worthy Claims, Previously Fact-Checked Claims, and Fake News, in: K. S. Candan, B. Ionescu, L. Goeuriot, B. Larsen, H. Müller, A. Joly, M. Maistro, F. Piroi, G. Faggioli, N. Ferro (Eds.), *Experimental IR Meets Multilinguality, Multimodality, and Interaction. Proceedings of the Twelfth International Conference of the CLEF Association (CLEF 2021)*, Lecture Notes in Computer Science (LNCS) 12880, Springer, Heidelberg, Germany, 2021, pp. 264–291.
- [10] G. Salton, *Automatic Information Organization and Retrieval.*, McGraw-Hill, New York, USA, 1968.